Lab 6 Guide: Deploying a Containerized Application on Cloud Run

Lab ID: CBL515
Level: Introductory
Time needed: About 1 hour
Cost: No cost

About this lab

Cloud Run runs stateless containers that respond to HTTP requests, without you managing any servers. Any language works as long as you can package it into a container image, and for Go, Node.js, Python, Java, .NET Core and Ruby, Cloud Run can even build the container from your source code for you. It is built on Knative, so the same containers can run fully managed on Cloud Run or inside your own Google Kubernetes Engine cluster.
In this lab you take a small Node.js retail API, run it locally in Cloud Shell, package it into a container image with Cloud Build, store the image in Artifact Registry, and deploy it to Cloud Run.

The build and deploy flow

Step	Where it happens	Result
1. Run the app locally	Cloud Shell	Confirms the API works before containerizing it
2. Build the image	Cloud Build	A Docker image built from the app's Dockerfile
3. Store the image	Artifact Registry (my-repo)	sample-node-app-image, ready to deploy
4. Deploy	Cloud Run	The sample-node-app service with a public HTTPS URL
5. Test	Cloud Shell	The same API responses, now served from Cloud Run

What you will learn

- How to build a Docker container image with Cloud Build
- How to store container images in an Artifact Registry repository
- How to deploy a container image to Cloud Run and test it

Before you begin

Start the lab

1. Sign in to Google Skills in an incognito window.
2. Check the lab's access time (for example, 1:15:00) and make sure you can finish within it. There is no pause; restarting takes you back to the beginning.
3. Select **Start lab** and note the **Username** and **Password** shown.
4. Select **Open Google Console**, choose **Use another account**, and sign in with the lab credentials. Other credentials will cause errors or charges.
5. Accept the terms and skip the recovery resource page.

Important: Don't select **End Lab** until you're finished. Ending the lab deletes the project and all your work.

Open Cloud Shell

Cloud Shell is a browser-based VM with development tools already installed, a persistent 5 GB home directory, and the gcloud CLI signed in to your lab project.

1. In the console's top-right toolbar, select **Open Cloud Shell**, then **Continue**.
2. Wait a few moments for the session to connect. Your project ID appears in the prompt.
3. Optionally, confirm which account and project you're using:

```
gcloud auth list
gcloud config list project
```

The first command should show your lab student account as active, and the second should show your qwiklabs-gcp project ID.

Task 1. Prepare the environment

You store your project ID and region in variables so later commands are shorter and less error-prone, then switch on the APIs this lab uses.

1. In Cloud Shell, set the variables. Replace [REGION] with the region shown in your Lab setup and access panel:

```
PROJECT_ID=$(gcloud config get-value project)
REGION=[REGION]
```

2. Enable the Artifact Registry, Cloud Build and Cloud Run APIs:

```
gcloud services enable artifactregistry.googleapis.com \
cloudbuild.googleapis.com \
run.googleapis.com
```

Tip: These variables only exist in the current Cloud Shell session. If you open a new session or your session restarts, run the two variable commands again before continuing.

Task 2. Run the app locally

Before containerizing anything, you check that the sample app works. It's a basic retail service whose API returns product data over HTTP.

Get the code

1. Create a working folder and move into it:

```
mkdir app && cd app
```

2. Copy the sample app from Cloud Storage and unzip it:

```
gsutil cp gs://cloud-training/CBL515/sample-apps/sample-node-app.zip
. && unzip sample-node-app
```

3. Move into the app folder and look at the main entry point, server.js:

```
cd sample-node-app
cat server.js
```

4. Install the app's dependencies:

```
npm install
```

Start the app and call the API

1. Start the server:

```
node server.js
```

You should see: **Sample-node-app listening on port 8080!** Leave it running.

2. Open a second Cloud Shell tab by selecting **Add** (the plus icon) in the Cloud Shell tab bar.
3. In the new tab, request the full product list. The jq tool formats the JSON response so it's easy to read.

```
curl http://localhost:8080/service/products | jq
```

You should get a JSON list of six products, including Home Barista Kit, Terrarium, Film Camera, Metal Camping Mug, City Bike and Air Plant. Each has an id, name, description, picture, cost and categories.

4. Request a single product by its ID:

```
curl http://localhost:8080/service/products/1YMWWN1N4O | jq
```

This time you get one product: the Home Barista Kit, cost 124, in the cookware category.

5. Go back to the first tab and press **Ctrl+C** to stop the server.

Task 3. Build the container image with Cloud Build

Cloud Build runs builds on Google Cloud. You describe the work in a build configuration file, and Cloud Build can fetch dependencies, run tests, and produce artifacts using tools such as docker, gradle or maven. It is often the build stage of a CI/CD pipeline. Here it builds a Docker image from the app's Dockerfile and pushes it to the Artifact Registry.

Create an Artifact Registry repository

1. In the console search bar, type **Artifact Registry** and open it. Optionally, pin it to the navigation menu with the pin icon.
2. Select **Create Repository** and enter the settings below, leaving everything else at the defaults.

Setting	**Value**
Name	my-repo

Format	Docker
Location type	Region
Region	The region from your Lab setup and access panel

3. Select **Create** and wait for the repository to appear.

Let Docker push to the repository

1. In Cloud Shell, configure Docker to authenticate with Artifact Registry in your region. Enter **Y** when prompted.

```
gcloud auth configure-docker ${REGION}-docker.pkg.dev
```

Write the build configuration

1. Store the repository path in a variable:

```
REPO=${REGION}-docker.pkg.dev/${PROJECT_ID}/my-repo
```

2. Create cloudbuild.yaml. Paste the whole block at once; the variable values are filled in as the file is written.

```
cat > cloudbuild.yaml <<EOF
steps:
- name: 'gcr.io/cloud-builders/docker'
  args: [ 'build', '-t', '${REPO}/sample-node-app-image', '.' ]
images:
- '${REPO}/sample-node-app-image'
EOF
```

3. Check the file. The image paths should now show your real region and project ID.

```
cat cloudbuild.yaml
```

What the file does: the single step uses the docker builder to build an image from the source code and Dockerfile in the current folder and tag it with the repository path. The images section tells Cloud Build to push that image to Artifact Registry when the build succeeds.

Run the build

1. Submit the build:

```
gcloud builds submit --region=$REGION --config=cloudbuild.yaml
```

2. Wait for the build to finish. The output ends with the image layers being pushed, then a summary showing the build ID, duration, image path and **STATUS: SUCCESS**. It usually takes one to two minutes.

Behind the scenes, Cloud Build uploads your source folder to Cloud Storage, builds the image using cloudbuild.yaml and the Dockerfile, tags it, and pushes it to my-repo.

Look at the results in the console

1. Search for **Cloud Build** in the console and open it. Set **Region** to your lab region.
2. Select the build ID at the top of the list to see the build details and full log.

3. Go to **Artifact Registry** > **Repositories** and open **my-repo**. You should see **sample-node-app-image**.

Task 4. Deploy the image to Cloud Run

Cloud Run can run containers in two ways:

Option	What it means
Cloud Run (fully managed)	Cloud Run handles the whole container lifecycle. This is what you use in this lab.
Cloud Run on Anthos / GKE	Run on your own GKE clusters for extra control over the underlying infrastructure.

1. In Cloud Shell, deploy the image as a service called sample-node-app:

```
gcloud run deploy sample-node-app --image
${REPO}/sample-node-app-image --region $REGION
--allow-unauthenticated
```

The --allow-unauthenticated flag makes the service publicly reachable without signing in, which is what you want for a public API.

2. When the command finishes, open **Cloud Run** from the navigation menu and select **sample-node-app** to see its details, including its URL.

Task 5. Test the app on Cloud Run

1. Confirm the service is running:

```
gcloud run services list
```

The output lists sample-node-app with its region, an HTTPS URL ending in run.app, and who deployed it and when.

2. Save the service URL in a variable and print it:

```
SERVICE_URL=$(gcloud run services describe sample-node-app
--region=$REGION --format='value(status.url)'); echo
"SERVICE_URL=$SERVICE_URL"
```

3. Call the products endpoint on Cloud Run:

```
curl ${SERVICE_URL}/service/products | jq
```

You should get the same six products you saw when running the app locally, now served by Cloud Run.